TEST REPORT

Summary and closure of the testing: web shop on Kubernetes

Project / system	Web shop on Kubernetes: four Python/Flask microservices with SQL Server
Release / version	1.1.0 (build 1.1.0-33b582d2e4e4)
Test period	2026-11-06
Document owner	Jakub Samborski
Date	2026-11-06
Status	Final

Test cases, test results and traceability to the requirements are in docs/test-management.xlsx. The automated tests are in tests/ and are run with python run.py.

1. Summary

This report summarizes the testing of the web shop: four Python/Flask microservices with a SQL Server database, deployed on Kubernetes (Docker Desktop). Testing was based on 41 requirements and risk. 103 test cases at four levels (unit, integration, end-to-end and static) focus on the checkout and order flow (no overselling, all-or-nothing orders, no double orders), accounts and security.

Result: All 103 test cases passed: 102 automated and 1 manual review. The automated suite of 208 tests passed in a full run from an empty environment, and all 41 requirements are covered and passed.

Defects: A review of version 1.0 found 20 defects, 4 of them Critical (for example passwords stored in plain text, and stock that could be oversold). All are fixed in version 1.1.0 and verified; none is open.

Remaining risk: No load testing, a single test environment, and the system is not hardened for production use (for example no HTTPS).

Recommendation: GO for use as a local demonstration system. Not for production use without the actions in section 10.

2. Scope

Testing covered the whole system as deployed by deploy.py. The priority of an area is the highest priority among its requirements.

Area / flow	Test level / test type	Priority	Comment
Checkout and orders	Unit, integration, E2E / functional, concurrency	Critical	All-or-nothing orders across services, no overselling with 20 parallel requests, idempotent checkout (double-click), order limits, delivery fee.
Accounts: register, log in, log out, delete	Unit, integration, E2E / functional, security	Critical	Boundary values for password, email and name lengths; equivalence classes for email; unique email also with simultaneous registrations; password hash checked in the database.
Catalogue and cart	Unit, E2E / functional	Critical	The cart can never hold more than the stock, including what is already in it; amounts exact to the cent.
Security	Unit, integration, E2E, static / security	Critical	Service token, network isolation between pods, database permissions per service, CSRF, rate limiting, XSS, error details, security headers, no credentials in the repository.
Operations and resilience	Integration / non-functional	High	Data survives a database restart; behaviour during a database outage; repeatable deployment; requests traceable across services.

Out of scope: Load and performance testing, browsers other than Chromium, real payment and e-mail providers, penetration testing, accessibility and usability testing, and testing in a production environment.

3. Tests executed

Every planned test case was executed. The automated tests run in one pytest run; the static review was done by hand.

Test area	Planned	Executed	Status / comment
Unit tests (UT)	40	40	110 automated tests, all passed. Services tested in isolation with a faked database and faked services.
Integration tests (IT)	41	41	77 automated tests against the deployed system, all passed, including database restart, outage and redeploy.
E2E tests (E2E)	21	21	21 Playwright tests in Chromium, all passed.
Static review (ST)	1	1	Manual review of code and configuration for credentials. Passed.
Total	103	103	100 % executed: 208 automated tests and 1 manual review.

Requirement coverage: All 41 requirements have at least one test case. Critical requirements are tested at several levels: for example REQ-22 (a user cannot order more than the stock allows) at unit, integration and E2E level.

Test design techniques: boundary value analysis, equivalence partitioning, decision tables, state transitions, concurrency tests and negative tests.

4. Results

Measure	Result	Comment
Test cases executed	100 % (103 of 103)	102 automated, 1 manual review.
Pass rate	100 % (103 of 103)	No failed or blocked test cases in the final run.
Automated tests	208 of 208 passed	Unit 110, integration 77, E2E 21. Run time about 5 minutes.
Requirements	41 of 41 passed	Every requirement covered by at least one passed test case.
Regression after the last change	208 of 208 passed	Full suite run again after the delivery price change (section 6).
Run in a clean environment	Passed	From an empty cluster: new secrets, new database, deploy, all tests, cleanup.

Measure	Result	Comment
Open Critical / High defects	0	All 20 defects from the review of version 1.0 are fixed.

Analysis: A pass rate of 100 % means that version 1.1.0 behaves as the 103 test cases specify. It does not mean the system is free of defects. The 20 defects in section 5 were found earlier, in the review of version 1.0, and test cases were written to confirm the fixes. To check that the tests really detect defects, four bugs were put into the code on purpose, one at a time: the unit tests failed for all four. The results are limited to one environment and do not say anything about behaviour under load (section 8).

5. Defects

No product defects were found while testing version 1.1.0. The defects below were found in the review (static testing) of version 1.0, fixed in version 1.1.0, and are verified: 17 by test cases, 3 by code review.

Severity	Total	Open	Comment / impact
Critical	4	0	Passwords stored in plain text; debug mode on in the containers (risk of remote code execution); credentials in the source code and one admin login for all services; stock could be oversold by simultaneous orders. Verified by UT-008, IT-008, IT-017, IT-031, ST-001; debug mode by code review.
High	6	0	Stock lost when saving an order failed; receipt said 'payment successful' when nothing was ordered; one checkout became several independent orders; duplicate email addresses possible; internal APIs open to anything in the cluster; no CSRF protection. Verified by UT-023, E2E-018, IT-016, IT-002 to IT-004, IT-025, IT-027, IT-032.
Medium	6	0	Invalid input crashed services (HTTP 500); error responses showed internal details; a double-click created two orders; unlimited login attempts; amounts calculated as floating point; orders pointed at deleted users. Verified by UT-021, UT-036, IT-020, UT-034, UT-016, IT-010.
Low	4	0	Delivery fee and post code ignored at checkout; cart accepted negative quantities; wrong default service addresses; database connections not closed after errors. Verified by E2E-016 and UT-011; the last two by code review.

Test code: Three defects in the test code itself were also found and fixed during the test runs (cookie handling, a database tool setting, recovery from dropped connections to the cluster). They did not affect the product.

6. Deviations from plan

The deviations below changed how the testing was done; none of them reduced the scope.

Deviation	Impact on testing	Action / handling

Deviation	Impact on testing	Action / handling
The original test scripts were replaced and testing restarted from the requirements	The old scripts reported PASS even when checks failed, so their results could not be used.	Requirements and test cases were written first, then a new pytest suite was built per test level.
Change request during testing: pickup at the store free, delivery to the door $5.00	REQ-20, E2E-016 and E2E-020 had to change.	Test cases and tests updated; full regression (208 of 208) and the clean-environment run repeated.
Unstable port-forwarding to the cluster	In two runs, 1 to 7 integration tests failed although the services worked.	The test code restarts dropped connections and repeats only requests that are safe to repeat. Later runs were stable.
Rate limit on login (5 per minute)	Automated logins in every test would have blocked the suite.	Tests that are not about login start with a signed session. Login tests use the form.
E2E-021 could not be executed as written	The browser refuses an email containing <script> in the login form.	Test case changed: the test helper logs in. What is checked (the email shown as text) is unchanged.

7. Risks

Risks identified during the testing and how they were handled.

Risk	Consequence	Action	Status
False failures caused by the test environment	The product could be judged faulty when it is not, and real failures could be hidden.	Self-healing connections; kubectl logs kept; every failure analysed before re-running.	Handled
Tests affecting each other through the shared database	Results depend on test order or on earlier runs.	Each test creates its own users, sets stock to a known value and restores it.	Handled
Disruptive tests (database restart, redeploy)	Other tests fail at random while the database is down.	Marked disruptive and always run last.	Handled
Tests that can never fail	False confidence in the results.	Four bugs put into the code on purpose; all were detected.	Handled
Cluster running an older version than the code	Testing a different version than intended.	The image tag is a hash of the source code, so every code change gives a new build.	Handled

8. Remaining risk

Risks and uncertainties that remain after the testing.

Remaining risk	Consequence	Risk level	Risk owner / accepted by
No load or performance testing	Response times and stability under real load are unknown.	Medium	Jakub (project owner)
Not hardened for production: no HTTPS, self-signed database certificate, one shared token between the services	Acceptable on a local machine, not on a public server.	Medium (High if deployed publicly)	Jakub (project owner)
Tested in one environment: Docker Desktop (kind) and Chromium	Other clusters or browsers may behave differently.	Low	Jakub (project owner)
Rate limit counted per pod	With two frontend pods, up to 10 login attempts per minute are possible.	Low	Jakub (project owner)
Payment and e-mail are mocked	Real integrations are not tested.	Low	Jakub (project owner)
7 rare code paths without a test case (e.g. registration when user-service fails between two calls)	Unlikely errors could go unnoticed.	Low	Jakub (project owner)

9. Conclusion

Testing shows that version 1.1.0 meets all 41 requirements in the tested environment. The critical flows work as intended: the stock can never be oversold, an order is placed completely or not at all, a double submission creates one order, and accounts and the internal services are protected. All defects found in the review of version 1.0 are fixed and verified. All exit criteria in scope are met. The main limitation is that nothing is known about behaviour under load.

Exit criterion	Status	Comment
All planned test cases executed	Met	103 of 103.
No open Critical or High defects	Met	0 open.
Every requirement covered by a passed test case	Met	41 of 41.
Fixed defects verified	Met	All 20: 17 by test cases, 3 by code review.
Full regression after the last change	Met	208 of 208.
The system and tests run in a clean environment	Met	python run.py from an empty cluster.

Exit criterion	Status	Comment
Performance requirements verified	Out of scope	No load test (remaining risk 1).
Test report written	Met	This report.

10. Recommendation

Recommendation	GO for use as a local demonstration system
Motivation	All exit criteria in scope are met, no defects are open, and the full regression and the clean-environment run passed. The remaining risks are documented and are Low or Medium for the intended use.
Conditions / actions	Before any production use: load and performance testing; HTTPS and a valid database certificate; a separate identity per service; tests against real payment and e-mail providers; a shared store for the rate limit.
Responsibility / follow-up	Jakub (project owner), 2026-11-06

11. Approval / recipients

Roles that have read or approved this report.

Role	Name	Date	Comment
Tester / report author	Jakub	2026-11-06	Report owner